

Event Stream Processor

You're working at an ad tech platform tracking ad events from publishers. The event streams contain duplicates, out-of-order timestamps, and require deduplication and aggregation.

Build an event processor that deduplicates events by ID (keeping the earliest timestamp), filters by event type, aggregates events into time windows, and yields results in batches using a generator.

Example 1 Input:

```
events = [  
    ("click_1", 1000, "click", "ad_banner_top"),  
    ("imp_1", 1001, "impression", "ad_sidebar"),  
    ("click_1", 999, "click", "ad_banner_top"),  
    ("imp_2", 1002, "impression", "ad_footer"),  
]  
event_types = None  
window_seconds = None  
batch_size = None
```

Output:

```
[  
    ("click_1", 999, "click", "ad_banner_top"),  
    ("imp_1", 1001, "impression", "ad_sidebar"),  
    ("imp_2", 1002, "impression", "ad_footer")  
]
```

Explanation: The event "click_1" appears twice with timestamps 1000 and 999. We keep the earliest (999). All events are deduplicated and sorted by timestamp. Since batch_size is None, the generator yields all results in a single batch.

Example 2 Input:

```
events = [  
    ("e1", 1000, "impression", "ad_A"),  
    ("e2", 1002, "click", "ad_A"),  
    ("e3", 1005, "impression", "ad_B"),  
    ("e4", 1008, "click", "ad_B"),  
    ("e5", 1010, "impression", "ad_C"),  
]  
event_types = None  
window_seconds = 5  
batch_size = None
```

Output:

```
[
  {
    'window_start': 1000,
    'window_end': 1004,
    'event_count': 2,
    'event_types': {'impression': 1, 'click': 1},
    'events': [("e1", 1000, "impression", "ad_A"), ("e2", 1002, "click", "ad_A")]
  },
  {
    'window_start': 1005,
    'window_end': 1009,
    'event_count': 2,
    'event_types': {'impression': 1, 'click': 1},
    'events': [("e3", 1005, "impression", "ad_B"), ("e4", 1008, "click", "ad_B")]
  },
  {
    'window_start': 1010,
    'window_end': 1014,
    'event_count': 1,
    'event_types': {'impression': 1},
    'events': [("e5", 1010, "impression", "ad_C")]
  }
]
```

Explanation: * Window [1000, 1004]: Contains e1 (1000) and e2 (1002) * Window [1005, 1009]: Contains e3 (1005) and e4 (1008) * Window [1010, 1014]: Contains e5 (1010) * Window calculation: $\text{window_start} = (\text{timestamp} // \text{window_seconds}) * \text{window_seconds}$ * Only windows containing events are included in the output. * Since batch_size is None, the generator yields all windows in a single batch.

Note: Events are not necessarily sorted by timestamp

Function Parameters * **events**: list of tuples (event_id, timestamp, event_type, data) * **event_types**: set of event types to include (None means all) * **window_seconds**: if provided, group events into time windows and aggregate (windows with no events are not included) * **batch_size**: size of batches to yield (None means yield all results in a single batch).

Returns * Generator yielding list batches

Constraints * $1 \leq \text{number of events} \leq 10^5$ * $1 \leq \text{timestamp} \leq 10^9$ * $1 \leq \text{length of event_id} \leq 50$ * $1 \leq \text{length of event_type} \leq 20$ * $1 \leq \text{length of data} \leq 100$ * $1 \leq \text{window_seconds} \leq 3600$ * $1 \leq \text{batch_size} \leq 1000$ * All strings contain only ASCII characters

Input Format for Custom Testing Input from stdin will be processed as follows and passed to the function:

First Line: List of events as tuples (JSON format) Second Line: Set of event types to filter (or None for all types) Third Line: Window size in seconds (or None for no windowing) Fourth Line: Batch size (or None for no batching)